

CSA-0301 — DATA STRUCTURES USING C

Concept Mapping – C Programming Solutions (Q1–Q15)

Name	Vishal Balaji J
Register No.	192525014
Course Code	CSA-0301

Each answer uses the data-structure idea requested in the question and includes a compact C implementation.

Q1 — Smart Healthcare: Data → Object → Structure → ADT → C

Concept / justification: Patient readings are primitive data; Patient is a structure/object; PatientADT hides storage and exposes operations. Arrays give simple indexed storage, while pointers allow efficient passing of records.

C Program

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char name[30];
    float temperature;
    char history[60];
    char diagnosis[60];
} Patient;

typedef struct { Patient data[100]; int n; } PatientADT;

void addPatient(PatientADT *a, Patient p) { a->data[a->n++] = p; }
void display(PatientADT *a) {
    for(int i=0;i<a->n;i++)
        printf("%d %s %.1f %s %s\n", a->data[i].id, a->data[i].name,
            a->data[i].temperature, a->data[i].history, a->data[i].diagnosis);
}

int main() {
    PatientADT db = {.n=0};
    Patient p={1,"Arun",98.6,"No allergy","Normal"};
    addPatient(&db,p); display(&db);
    return 0;
}
```

Q2 — Smart City Traffic: Primitive → Non-Primitive → Linear/Non-Linear

Concept / justification: Vehicle counts/signal values are primitive. Queues and linked lists model linear traffic streams; graphs model road networks, where junctions are vertices and roads are edges.

C Program

```
#include <stdio.h>
#define N 4
int graph[N][N]={0,1,1,0},{1,0,1,0},{1,1,0,1},{0,0,1,0}};
int queue[N], front=0, rear=0, visited[N];

void bfs(int s){
    queue[rear++]=s; visited[s]=1;
    while(front<rear){
        int u=queue[front++]; printf("%d ",u);
        for(int v=0;v<N;v++){
            if(graph[u][v] && !visited[v]) {
                visited[v]=1; queue[rear++]=v;
            }
        }
    }
}

int main(){ printf("Reachable junctions: "); bfs(0); return 0; }
```

Q3 — E-Commerce: Linear Structures → Sequential Access → Non-Linear

Concept / justification: Arrays/lists are suitable for sequential product storage. Trees support hierarchical searching; graphs can model recommendation relationships. Non-linear structures are better for complex relationships.

C Program

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int id; struct Node *left,*right; } Node;

Node* insert(Node* r,int x){
    if(!r){ Node*n=malloc(sizeof(Node)); n->id=x;n->left=n->right=NULL;return n; }
    if(x<r->id) r->left=insert(r->left,x); else r->right=insert(r->right,x);
    return r;
}
void search(Node*r,int x){
    if(!r) printf("Not found");
    else if(r->id==x) printf("Product found");
    else search(x<r->id?r->left:r->right,x);
}
int main(){ Node*r=NULL; int a[]={40,20,60,10,30}; for(int i=0;i<5;i++)r=insert(r,a[i]); search(r,30); }
```

Q4 — Ride Sharing: Algorithm → Input Size → Time/Space Complexity

Concept / justification: A linear scan takes $O(n)$ time and $O(1)$ auxiliary space. As driver count grows, execution time grows linearly. Spatial trees/graphs can improve large real-time systems.

C Program

```
#include <stdio.h>
typedef struct { int id, distance; } Driver;
int main(){
    Driver d[]={ {1,8}, {2,3}, {3,6}, {4,2} };
    int n=4,best=0;
    for(int i=1;i<n;i++)
        if(d[i].distance<d[best].distance) best=i;
    printf("Nearest driver: %d, distance: %d km",d[best].id,d[best].distance);
    return 0;
}
```

Q5 — Mobile App: Memory Usage → Space Complexity → Auxiliary Space

Concept / justification: The input array uses $O(n)$ memory, but iterative binary search uses $O(1)$ auxiliary space and $O(\log n)$ time, making it suitable when memory is constrained.

C Program

```
#include <stdio.h>
int binarySearch(int a[],int n,int key){
    int l=0,r=n-1;
    while(l<=r){
        int m=l+(r-l)/2;
        if(a[m]==key)return m;
        if(a[m]<key)l=m+1; else r=m-1;
    }
    return -1;
}
int main(){
    int a[]={2,5,8,12,16,21};
    printf("Index = %d",binarySearch(a,6,16));
    return 0;
}
```

Q6 — Banking ADT: ADT → Operations → Interface → Implementation

Concept / justification: The ADT exposes deposit, withdraw and balance operations while hiding implementation details. This separation improves modularity, reuse and maintainability.

C Program

```
#include <stdio.h>
typedef struct { int accNo; double balance; } Account;

void deposit(Account *a, double x){ if(x>0)a->balance+=x; }
int withdraw(Account *a, double x){
    if(x>0 && x<=a->balance){a->balance-=x;return 1;} return 0;
}
double getBalance(const Account *a){ return a->balance; }

int main(){
    Account a={1001,5000};
    deposit(&a,1000); withdraw(&a,750);
    printf("Balance: %.2lf",getBalance(&a));
    return 0;
}
```

Q7 — File System: Recursion → Base Case → Call Stack

Concept / justification: Each recursive call handles one folder and calls itself for child folders. Calls are stored on the call stack; depth determines stack memory. Recursion naturally matches hierarchical data.

C Program

```
#include <stdio.h>
typedef struct Folder {
    char name[20];
    struct Folder *child[3];
    int count;
} Folder;

void traverse(Folder *f,int level){
    if(f==NULL) return;           // base case
    for(int i=0;i<level;i++) printf(" ");
    printf("%s\n",f->name);
    for(int i=0;i<f->count;i++) traverse(f->child[i],level+1);
}

int main(){
    Folder a={"Root",{0},2}, b={"Docs",{0},0}, c={"Pics",{0},0};
    a.child[0]=&b; a.child[1]=&c; traverse(&a,0); return 0;
}
```

Q8 — Recursion vs Iteration: Call Stack → Memory vs Loop Control

Concept / justification: Both take $O(n)$ time. Recursive factorial uses $O(n)$ stack space; iteration uses $O(1)$ auxiliary space. Recursion is clearer for nested structures, while iteration is usually more memory-efficient.

C Program

```
#include <stdio.h>
long long factRec(int n){ return n<=1?1:n*factRec(n-1); }
long long factIter(int n){
    long long f=1;
    for(int i=2;i<=n;i++) f*=i;
    return f;
}
int main(){
    printf("Recursive=%lld\nIterative=%lld", factRec(5), factIter(5));
    return 0;
}
```

Q9 — Music Playlist: Array → Linked List → Stack → Queue

Concept / justification: A linked list supports flexible insert/delete of songs. A queue models play order (FIFO), while a stack can model recently played/undo operations. Arrays offer fast indexing but costly middle insertion.

C Program

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Song { char title[30]; struct Song *next; } Song;

void add(Song **head, char *t){
    Song*n=malloc(sizeof(Song)); snprintf(n->title,30,"%s",t); n->next=NULL;
    if(!*head){*head=n;return;}
    Song*p=*head; while(p->next)p=p->next; p->next=n;
}

void play(Song*h){ while(h){printf("Playing: %s\n",h->title);h=h->next;} }

int main(){ Song*p=NULL; add(&p,"Song A");add(&p,"Song B");add(&p,"Song C");play(p); }
```

Q10 — Social Network: Tree → Graph → DFS/BFS

Concept / justification: A tree gives only hierarchical parent-child relations, whereas a graph supports many-to-many friendships/follows. DFS/BFS efficiently traverses connected users, so graphs better represent social networks.

C Program

```
#include <stdio.h>
#define N 5
int g[N][N]={0,1,1,0,0},{1,0,0,1,0},{1,0,0,1,1},{0,1,1,0,0},{0,0,1,0,0}};
int vis[N];
void dfs(int u){
    vis[u]=1; printf("%d ",u);
    for(int v=0;v<N;v++) if(g[u][v]&&!vis[v]) dfs(v);
}
int main(){ printf("DFS users: "); dfs(0); return 0; }
```

Q11 — Searching: Linear Search → Binary Search → Complexity

Concept / justification: Linear search is $O(n)$ and works on unsorted data. Binary search is $O(\log n)$ but requires sorted, indexable data. For a large sorted dataset, binary search is more efficient.

C Program

```
#include <stdio.h>
int linear(int a[],int n,int k){
    for(int i=0;i<n;i++) if(a[i]==k)return i; return -1;
}
int binary(int a[],int n,int k){
    int l=0,r=n-1;
    while(l<=r){int m=(l+r)/2;if(a[m]==k)return m;
        if(a[m]<k)l=m+1;else r=m-1;}
    return -1;
}
int main(){
    int a[]={10,20,30,40,50,60,70};
    printf("Linear index=%d\nBinary index=%d",linear(a,7,60),binary(a,7,60));
}
```

Q12 — Ride Sharing Integration: ADT → Linear/Non-Linear → Algorithms

Concept / justification: Requests can be held in queues, drivers in arrays/lists, and routes in graphs. Algorithms combine these ADTs to allocate drivers and select routes. Choosing efficient structures supports scalability.

C Program

```
#include <stdio.h>
#define N 4
typedef struct { int userId, source, destination; } Request;
int roads[N][N]={0,2,0,6},{2,0,3,0},{0,3,0,1},{6,0,1,0}};

int nearestConnected(int source){
    int best=-1,min=999;
    for(int i=0;i<N;i++)
        if(roads[source][i]>0 && roads[source][i]<min){min=roads[source][i];best=i;}
    return best;
}

int main(){
    Request q={101,0,3};
    printf("Request %d: nearest connected node=%d",q.userId,nearestConnected(q.source));
}
```

Q13 — Library: Data → Structure → Organization → Access → Efficiency

Concept / justification: Structures group book fields; arrays store many records. For thousands of books, hashing can give average $O(1)$ lookup and balanced trees $O(\log n)$, improving retrieval over $O(n)$ sequential scans.

C Program

```
#include <stdio.h>
#include <string.h>
typedef struct { int id; char title[40],author[30]; int available; } Book;
int main(){
    Book b[]={1,"C Programming","Dennis",1},{2,"Data Structures","Mark",0},{3,"Algorithms","Sara",1}};
    char key[]="Data Structures";
    for(int i=0;i<3;i++){
        if(strcmp(b[i].title,key)==0){
            printf("%s by %s - %s",b[i].title,b[i].author,b[i].available?"Available":"Issued");
            break;
        }
    }
    return 0;
}
```

Q14 — Ticket Booking ADT: Operations → Implementation → Performance

Concept / justification: An array provides $O(1)$ seat-status access and is ideal when seat numbers are fixed. Linked lists are flexible but require traversal. For real-time seat allocation, indexed arrays/bitsets are typically faster.

C Program

```
#include <stdio.h>
#define SEATS 10
typedef struct { int seat[SEATS]; } BookingADT;
int book(BookingADT*b,int s){
    if(s<0||s>=SEATS||b->seat[s])return 0;
    b->seat[s]=1; return 1;
}
void cancel(BookingADT*b,int s){ if(s>=0&& s<SEATS)b->seat[s]=0; }
int main(){
    BookingADT b={{0}};
    printf("%s\n",book(&b,4)?"Seat booked":"Unavailable");
    cancel(&b,4); printf("Seat 5 cancelled");
}
```

Q15 — Weather Monitoring: Primitive → Non-Primitive → Linear → Non-Linear

Concept / justification: Temperature, humidity and wind are primitive numeric values grouped into Reading structures. A circular queue handles continuous time-ordered sensor data; trees/graphs can organize stations and geographic relationships.

C Program

```
#include <stdio.h>
#define MAX 5
typedef struct { float temp,humidity,wind; } Reading;
typedef struct { Reading q[MAX]; int front,rear,count; } SensorQueue;

void enqueue(SensorQueue*s,Reading r){
    if(s->count==MAX){s->front=(s->front+1)%MAX;s->count--;}
    s->q[s->rear]=r; s->rear=(s->rear+1)%MAX;s->count++;
}

int main(){
    SensorQueue s={0};
    enqueue(&s,(Reading){31.5,70,12});
    enqueue(&s,(Reading){32.0,68,15});
    printf("Stored readings: %d",s.count);
    return 0;
}
```